03–03 – Porucha s opravou, návrat do fronty

Zadání

> Uvažujme systém typu M/M/1. U linky dochází k poruše v intervalech exponenciálně se středem E časových jednotek. Porucha zabere linku na dobu R. Pokud je v okamžiku vzniku poruchy nějaká transakce v obsluze, pak je její obsluha zrušena a transakce se vrací do fronty. Nakreslete model systému s použitím Petriho sítí.

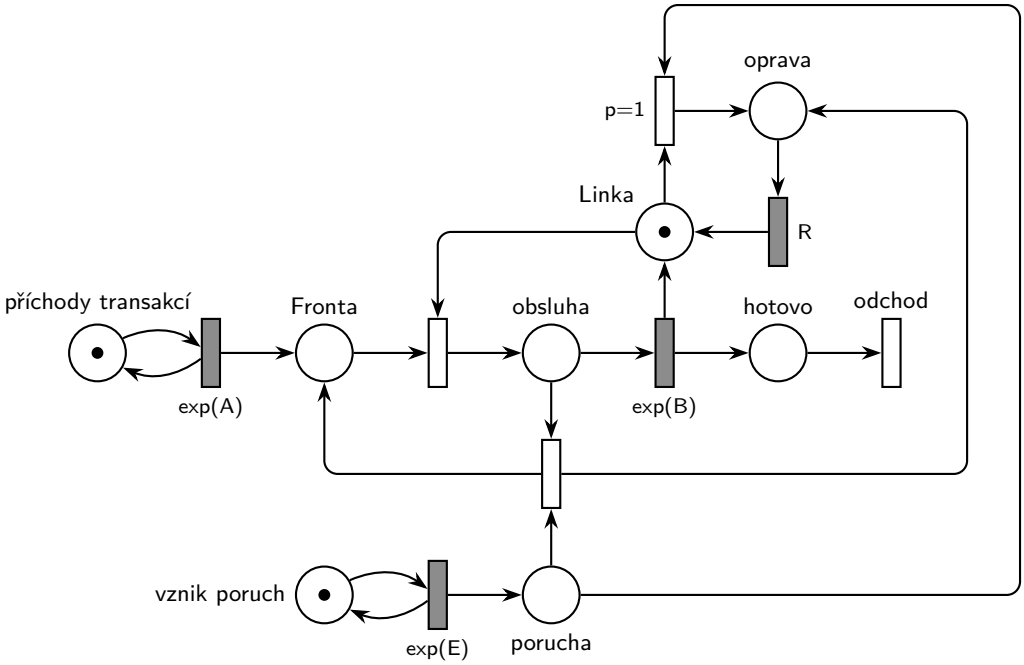
Podobné úlohy:

- 03–02 – Porucha vrací transakci do fronty: totéž bez doby poruchy
- 03–04 – Porucha s opravou, transakce odchází a 03–05 – Kritická havárie: stejná síť, jen přerušená transakce odchází
- 03–07 – Každá 10. transakce vyvolá poruchu: stejné zabrání linky na dobu poruchy, spouští ho čítač

Interpretace

- **M/M/1:** *příchody transakcí* ve smyčce s `exp(A)`, *Fronta*, obsazení *Linky*, obsluha `exp(B)`, která *Linku* vrací, a *odchod*. Časy příchodů a obsluhy zadání neuvádí, proto `exp(A)` a `exp(B)`.
- **Poruchy** vznikají v intervalech `exp(E)`: místo *vznik poruch* ve smyčce s časovaným přechodem, každé provedení vloží značku do místa *porucha*.
- **„Porucha zabere linku na dobu R“:** porucha se chová jako transakce s vyšší prioritou obsluhy (v SIMLIBu `Seize(Linka, 1)`): linka je po dobu R v místě *oprava* a časovaný přechod R ji pak vrátí.
- **Volná linka:** přechod p=1 nahoře spojí *poruchu* s *Linkou* a začne oprava. Priorita je nutná: porucha, která čekala během předchozí opravy, musí linku po jejím skončení dostat dřív než transakce čekající ve *Frontě* (obyčejné obsazení je v tu chvíli proveditelné také).
- **Obsazená linka:** přechod dole vezme *poruchu* a obsluhovanou transakci (místo *obsluha*), transakci vrátí do *Fronty* a linku předá rovnou do *opravy*: značka *Linky* je „v“ obsluhované transakci (*Linka* + *obsluha* + *oprava* = 1), takže ji porucha převezme bez mezikroku. Přerušení funguje díky sémantice časovaného přechodu (slajd 139): okamžitý přechod si vezme značku z místa *obsluha* a odpočet `exp(B)` zanikne.
- Stejně správná je varianta ve dvou krocích: přerušení vrátí značku *Lince* a poruchu nechá v místě *porucha* (hrana tam a zpět), hned potom ji přechod p=1 spojí s volnou *Linkou*. Tady je kreslená v jednom kroku, protože síť je jednodušší.
- **Porucha během opravy:** linka je zabraná, porucha tedy počká v místě *porucha* a linku zabere hned po skončení opravy (další doba R). Stejně by se chovala porucha v SIMLIBu, která čeká ve frontě k zařízení. Jiné rozumné čtení je, že porucha během opravy zanikne (rozbitá linka se znovu nerozbije). Pak by stačilo přidat přechod *porucha bez účinku* s `p=0` jako v 03–02: provede se jen tehdy, když porucha nemůže linku přerušit ani zabrat, tedy právě během opravy.
- Kam do fronty se transakce vrací (na začátek, nebo na konec), Petriho síť nerozliší. Obsluhu začne znovu od začátku.
- E a R jsou parametry ze zadání. Doba poruchy R je konstanta (zadání neříká, že je náhodná).

Petriho síť



Časté chyby

- Porucha jen přeruší obsluhu, ale linku nezabere: chybí doba R (to je úloha 03–02).
- Porucha zabere jen volnou linku a obsazenou ignoruje, nebo naopak volnou linku nezabere a porucha zanikne.
- Zabrání volné linky bez priority: porucha čekající během opravy prohraje s transakcí z *Fronty* a oprava se odloží.
- Linka se po opravě nevrátí (chybí hrana z R do *Linky*).
- Přerušená transakce poslaná na odchod místo do *Fronty*.
- V SIMLIBu jen `Seize(Linka, 1)`: přerušená transakce zůstane ve frontě přerušených a po opravě obsluhu **dokončí** (zbývající čas), místo aby se vrátila do fronty a začala znovu.

SIMLIB

Model je izomorfní k Petriho síti výše: každé místo, přechod a značka mají svůj protějšek v kódu.

```
// 03–03 Porucha s opravou, návrat do fronty – model v SIMLIB/C++
#include "simlib.h"

const double A = 10, B = 6, E = 100, R = 5;  // A, B: symboly ze sítě (zadání je
 ↪  nemá)

Facility Linka("Linka");                  // místo Linka (zařízení s kapacitou 1)
Histogram Doba("Doba transakce v systému", 0, 10, 20);
long poruchy = 0, preruseno = 0;

class Transakce;
Transakce *obsluhovana = nullptr;        // místo „obsluha“ (nullptr = bez značky)

class Transakce : public Process {        // transakce = značka, která prochází
 ↪  sítí
 public:
  bool prerusena = false;
  void Behavior() {
    double prichod = Time;
    do {
```

```
      prerusena = false;
      Seize(Linka);                          // obsazení: Fronta + Linka → obsluha
      obsluhovana = this;
      Wait(Exponential(B));                  // časovaný přechod exp(B), porucha ho
 ↪  může přerušit
    } while (prerusena);                    // přerušená transakce se vrací do Fronty
    obsluhovana = nullptr;
    Release(Linka);                         // exp(B) vrací Linku
    Doba(Time - prichod);
  }                                         // konec Behavior = přechod „odchod“
};

class Porucha : public Process {            // porucha = značka v místě „porucha“
  void Behavior() {
    poruchy++;
    Transakce *t = obsluhovana;             // probíhající obsluha, nebo nullptr
    Seize(Linka, 1);                        // priorita obsluhy 1: přerušení obsazené
 ↪  Linky,
                                            // u volné Linky zabrání p=1 (předběhne
                                             ↪  Frontu)
    if (t) {                                // přechod přerušení: transakce zpět do
     ↪  Fronty
      t->Out();                             // SIMLIB by ji nechal ve frontě
 ↪  přerušených
      t->prerusena = true;                  // a po opravě by obsluhu jen dokončil
      t->Activate();
      obsluhovana = nullptr;
      preruseno++;
    }
    Wait(R);                                // místo „oprava“ + časovaný přechod R ...
    Release(Linka);                         // ... který vrací Linku
  }
};

class GeneratorPoruch : public Event {      // „vznik poruch“ ve smyčce s exp(E)
  void Behavior() {
    (new Porucha)->Activate();
    Activate(Time + Exponential(E));
  }
};

class Generator : public Event {            // „příchody transakcí“ ve smyčce s
 ↪  exp(A)
  void Behavior() {
    (new Transakce)->Activate();            // nová značka v místě „Fronta“
    Activate(Time + Exponential(A));
  }
};

int main() {
  Init(0, 1000000);                         // modelový čas v časových jednotkách sítě
  (new Generator)->Activate();
  (new GeneratorPoruch)->Activate(Exponential(E));
  Run();
  Linka.Output();
  Doba.Output();
  Print("Poruchy: %ld, z toho přerušily obsluhu: %ld\n", poruchy, preruseno);
}
```

Petriho síť	SIMLIB
příchody transakcí ⇄ `exp(A)`	událost `Generator`, která se znovu plánuje za `Exponential(A)`
značka (transakce)	proces `Transakce`
Fronta + obsazení *Linky*	`Seize(Linka)` (fronta je vnitřní fronta zařízení)
místo *Linka* (1 značka)	`Facility Linka`
místo *obsluha*	ukazatel `obsluhovana` (`nullptr` = místo je prázdné)
`exp(B)` se sloučeným uvolněním *Linky*	`Wait(Exponential(B)); Release(Linka);`
vznik poruch ⇄ `exp(E)`	událost `GeneratorPoruch`, která se znovu plánuje za `Exponential(E)`
značka v místě *porucha*	proces `Porucha`
zabrání volné *Linky* (p=1)	`Seize(Linka, 1)`: priorita obsluhy 1 řadí poruchu ve frontě zařízení před transakce
přerušení (porucha + *obsluha* → *oprava* + *Fronta*)	`Seize(Linka, 1)` u obsazené linky: SIMLIB vezme linku obsluhované transakci; `t->Out()` ji vyjme z fronty přerušených a `Activate()` s příznakem `prerusena` ji pošle cyklem `do … while` zpět k `Seize(Linka)`
místo *oprava* + časovaný přechod R	`Wait(R); Release(Linka);` v procesu `Porucha`
přechod *odchod*	konec metody `Behavior()` transakce

Kontrola simulací. Petriho síť: *Linka* + *obsluha* + *oprava* = 1 a všechny přechody se provádějí. Každá porucha linku zabere (počet provedení R = počet poruch), asi 60 % poruch přitom zruší obsluhu. Poruchy čekají na linku jen výjimečně (nejvýš dvě v místě *porucha*) (`make check`). SIMLIB model: v jednom běhu (10^6 časových jednotek, A = 10, B = 6, E = 100, R = 5) přerušilo obsluhu 6 042 z 10 008 poruch (60 %) a linka byla obsazená 65,6 % času (transakce i opravy). Teoreticky transakce linku vytíží na 0,1 × 6 = 60 % (exponenciální obsluha nemá paměť, opakování dají dohromady zase průměrně 6) a opravy na R/E = 5 %, celkem 65 % (`make simlib`).

Původ a zdroje

Původ: zkouška 2024/25, 1. termín, skupina A, otázka 3, 10 bodů.

Původní znění:

> Uvažujme systém typu M/M/1. U linky dochází k poruše v intervalech daných Exp(E). Porucha zabere linku na dobu R. Pokud je v okamžiku vzniku poruchy nějaká transakce v obsluze, pak je její obsluha zrušena a transakce se vrací do fronty. Nakreslete model systému s použitím Petriho sítí.

Zdroje:

- `Fitcheats/Zkoušky z minulých let/2024-25 - 1. termín A+C.pdf`, str. 2 (fotka zadání, rámeček na odpověď je prázdný)

Existující řešení:

- K tomuto zadání žádné řešení není. Nejbližší je **fifinas, str. 2** (úloha 03–04): porucha zabere volnou linku přechodem s p=1 a obsazenou linku přechodem, který vezme obsluhovanou transakci. Oba vedou do místa před časovaným přechodem C, který linku vrátí, stejně jako tady. Rozdíl je jen v tom, že tam přerušená transakce mizí, kdežto tady se vrací do *Fronty*.